Detecting Extreme Traffic Events Via a Context Augmented Graph Autoencoder

YUE HU, AO QU, and DAN WORK, Vanderbilt University

Accurate and timely detection of large events on urban transportation networks enables informed mobility management. This work tackles the problem of extreme event detection on large-scale transportation networks using origin-destination mobility data, which is now widely available. Such data is highly structured in time and space, but high dimensional and sparse. Current multivariate time series anomaly detection methods cannot fully address these challenges. To exploit the structure of mobility data, we formulate the event detection problem in a novel way, as detecting anomalies in a set of time-dependent directed weighted graphs. We further propose a *Context augmented Graph Autoencoder* (Con-GAE) model to solve the problem, which leverages graph embedding and context embedding techniques to capture the spatial and temporal patterns. Con-GAE adopts an autoencoder framework and detects anomalies via semi-supervised learning. The performance of the method is assessed on several city-scale travel-time datasets from Uber Movement, New York taxis, and Chicago taxis and compared to state-of-the-art approaches. The proposed Con-GAE can achieve an improvement in the area under the curve score as large as 0.15 over the second best method. We also discuss real-world traffic anomalies detected by Con-GAE.

CCS Concepts: • **Applied computing** → **Transportation**; • **Information systems** → *Spatial-temporal systems*; *Data mining;*

Additional Key Words and Phrases: Anomaly detection, transportation, graph neural network, autoencoder

ACM Reference format:
Yue Hu, Ao Qu, and Dan Work. 2022. Detecting Extreme Traffic Events Via a Context Augmented Graph Autoencoder. *ACM Trans. Intell. Syst. Technol.* 13, 6, Article 101 (September 2022), 23 pages.
https://doi.org/10.1145/3539735

1 INTRODUCTION

Extreme event detection has been a long-standing task in transportation science [34], and is becoming increasingly important for traffic management. When not handled efficiently, an extreme event might cause a large portion of the urban traffic network to be disrupted, instead of having only localized impacts that are common for minor incidents. Accurate and timely detection of such network-wide traffic anomalies can help with resilience analysis as well as real-time congestion mitigation, enabling informed urban traffic management. Therefore, the objective of this work is to

This material is based upon work supported by the National Science Foundation under Grant No. CMMI-1727785.
Authors' addresses: Y. Hu and A. Qu, Vanderbilt University, 1025 16th Ave. S., Nashville, Tennessee, USA, 37212; emails: {yue.hu, ao.qu}@vanderbilt.edu; D. Work, Vanderbilt University, Nashville, Tennessee, USA; email: dan.work@vanderbilt.edu.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2157-6904/2022/09-ART101 $15.00
https://doi.org/10.1145/3539735

detect extreme traffic events, which are anomalies that manifest on large portions of the network at once.

Today, abundant transportation data facilitates real time traffic monitoring and anomaly detection, with the aid of machine learning techniques. One promising area is in urban mobility, where large fleets of instrumented vehicles (e.g., ride sharing services and taxis) collect abundant traffic information, which is then processed into ***origin-destination* (OD)** travel-time data and made publicly available to assist urban mobility managers. Given a city that is spatially partitioned into different zones, an OD travel-time record describes the travel time from the trip origin zone to the trip destination zone. Examples include data products from Uber Movement [47], taxi datasets in Chicago [10] and New York City [46], and micro-mobility data from bike share and scooter operators (e.g., Reference [9]). The OD travel-time data is organized into an origin-zone by destination-zone matrix, which is oftentimes incomplete due to missing records.

Despite the growing availability of OD data, it is often challenging for urban mobility managers to directly use the data for performance monitoring and event detection. The challenge lies in **how to fully exploit the spatial-temporal correlations in the traffic OD matrix, and meanwhile overcome the high dimensionality and extreme sparsity** of common OD data matrices. In previous studies, robust matrix and tensor decomposition-based methods are used to separate traffic during extreme events from regular traffic [24, 43, 58]. These methods take advantage of the multi-dimensional correlation of regular traffic in space and time, but can only model the linear correlations of complex nonlinear traffic dynamics. With recent advances in deep learning-based multivariate time series anomaly detection, one straightforward solution is to model the traffic OD data as a multivariate time series, where the feature vector is composed by flattening all OD pairs. However, the currently available time series anomaly detection methods [2, 33, 62] cannot capture the spatial information of the traffic network, nor can they explicitly utilize the longer-term (e.g., daily and weekly) periodicity present in regular traffic, since they mainly rely on short-term fluctuations in time. Intuitively, learning the typical traffic conditions (e.g., Monday 7 a.m.) is more helpful to detect an anomaly than knowing the traffic conditions an hour before.

To address these challenges, we propose a ***Context augmented graph autoencoder* (Con-GAE)** that leverages graph embedding and context embedding techniques to capture the spatio-temporal traffic network patterns. We model the origin and destination zones as nodes in a graph. The spatial pattern is then captured by graph convolutional layers modified to account for asymmetry in travel times. The daily and weekly periodicity in the data is captured by a context embedding. Finally, Con-GAE adopts a hierarchical structure to aggregate the spatial and temporal embeddings of the traffic network at each time slot into a single comprehensive graph embedding, to detect time contextualized anomaly graphs as a whole. By design, our model works on very sparse data, since learning from partially observed OD pairs is essentially the same as learning the common message-passing rules of a ***graph convolutional network* (GCN)** from a subset of all edges. It is equivalent to mini-batch stochastic gradient descent learning, where we subsample from all samples for each update step. Moreover, we encode the geographical features of each node, which are more complete, to help estimate the travel time. We also note that our method bypasses the high dimensionality of the OD matrix, since we are not considering all N^2 entries at each step. Instead, we compute embeddings for each node and the weights of sparsely connected edges (in the order of $O(N + E)$) at each time, where N is the number of nodes and E is the number of edges.

We give a high-level overview of the structure that enables Con-GAE to fully capture the spatial correlations of travel times between zones. Intuitively, if two zones (e.g., zone A and zone B) are close to each other in terms of travel time (i.e., the travel time from zone A to zone B is small, and the travel time from zone B to zone A is small), then the travel time from A to any other zone should be similar to the travel time from B to any other zone. We consider physical closeness

by using geographic information as input node features, and consider the travel-time connection via edge weights. Then Con-GAE learns the vector representation of each node given both node and weight information, such that zone A and zone B have similar embeddings when they are physically close or close in travel time. The node embeddings are further used to decode the edge weights, so two origin zones with similar embeddings will lead to similar travel times to other nodes in the network.

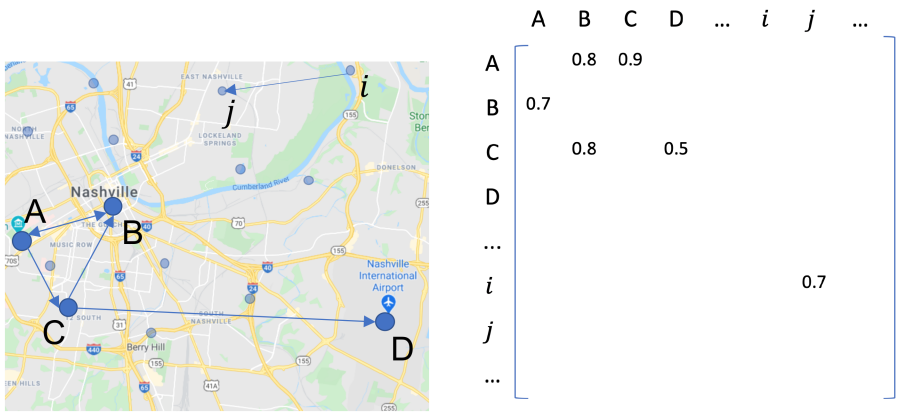
Our work is inspired by two threads of recent development in deep learning, namely, graph convolutional networks and autoencoder-based anomaly detectors. The graph convolutional layers in Con-GAE are a modification of the GCN [27]. Given a graph, GCN learns an embedding of the nodes that encodes structural information about the graph via aggregation of the embeddings of neighboring nodes on the graph. While modified GCN layers can capture the geometric structure of the network, the dynamic nature of traffic requires further consideration of the temporal dimension. Existing works on dynamic graphs [21, 68] mainly focus on modeling the short-term dependencies of consecutive graph instances. In comparison, we focus on exploiting the temporal periodicity of traffic graphs over longer time horizons such as daily or weekly patterns. We further emphasize that while some approaches detect anomalous nodes and edges in dynamic graphs [61, 65], our task of extreme event detection requires detecting the anomalous graph as a whole, rather than a part of a graph as anomaly.

Con-GAE determines anomalies via an ***autoencoder*** **(AE)** framework [38] for semi-supervised anomaly detection. An AE consists of an encoder to compress the high-dimensional input into a low dimensional space, and a decoder to map the low dimensional representation back to the original input space minimizing the reconstruction error. When trained only on normal data, the AE is assumed to encode and decode normal data well, but not anomalous data. Thus during testing, samples with large reconstruction errors are marked as anomalies. AE-based anomaly detectors have shown success in various high-dimensional anomaly detection tasks [11, 56, 66]. Here, we specialize the AE network architecture to exploit spatio-temporal structure in mobility data. We also note that as shown in Reference [7], in cases where we do not have access to clean training data, the reconstruction-based anomaly detector can also tolerate a small portion of anomalies in the training data. Because the training data is dominated by normal data, and the abnormal data is rare, the model will capture the normal patterns and is unable to learn the abnormal patterns in the data. We use experiments to show the robustness of our model to anomalies appearing in the training data.

We show the effectiveness of our model on several large-scale real-world datasets. We apply Con-GAE to OD travel-time datasets from Uber Movement, New York taxis and Chicago taxis, to detect two kinds of synthesized anomalies, i.e., large spatial anomalies impacting the network, and traffic conditions that are otherwise normal but not for the given time. Such temporal anomalies occur, e.g., due to holidays. We vary the magnitude of the anomalies and the number of anomalous instances, and show that our model outperforms several state-of-the-art baseline methods for high-dimensional or spatio-temporal anomaly detection. Finally, we show how the model reveals the influence of a real large-scale event contained in the dataset.

In summary, our contributions are as follows:

- We formulate the problem of anomaly detection in mobility OD datasets in a novel way, as detecting anomalies in a set of time-dependent directed weighted graphs. Each graph has geographic information contained in the node features and travel-time information in the edge weights.
- We propose Con-GAE, which is an autoencoder-based anomaly detector. Con-GAE uses GCN layers to generate node embeddings that capture spatial relationships, and time embeddings to capture the temporal periodicity.



(a) Traffic network graph (b) Weighted adjacency matrix

Fig. 1. Illustration of the OD mobility graph (left) and corresponding weighted adjacency matrix (right) corresponding to scaled inverse travel times between points on the graph. The weighted adjacency matrix can be highly sparse due to lack of data on all OD pairs.

- We perform experiments on OD mobility datasets, and demonstrate that Con-GAE outperforms several state-of-the-art anomaly detection methods.

2 METHODOLOGY

In this section, we first model the traffic network as a graph and formulate the anomaly detection problem. Then, we address the graph encoder and decoder structures. Finally, we address the loss function and regularization strategies used.

2.1 Modeling Mobility Data as a Graph

We summarize the origin destination matrix containing mobility data and introduce the graph model of this data. We then formulate the anomaly detection problem on the graph.

An OD matrix is a way to summarize a common type of transportation data in which travel times between different regions of a city are stored. Precisely, in our setting, the city is spatially partitioned into n different geographical *zones* (e.g., a neighborhood or a census tract). Each zone serves both as an origin zone in which trips begin, and a destination zone in which trips terminate. At a given time, there are a total of $n \times n$ travel times from each origin zone to each other destination zone (including trips that start and end in the same zone). The OD matrix, therefore, is an $n \times n$ matrix M in which the element $m_{i,j}$ denotes the travel time from origin zone i to destination zone j.

The OD data can be modeled as a set of time-dependent directed weighted graphs. At discrete time $t \in \{1, 2, \ldots, T\} = \mathcal{T}$, the graph is denoted as $G(t) = (\mathcal{V}, \mathcal{E}(t), \mathbf{W}(t))$. The node set $\mathcal{V}$ is time invariant, representing the $N = |\mathcal{V}|$ distinct geographical zones in the city. There is an edge $e_{ij} \in \mathcal{E}(t)$ if the travel-time information is available from node $v_i \in \mathcal{V}$ to $v_j \in \mathcal{V}$ at time t. The weighted adjacency matrix is denoted $\mathbf{W}(t) \in \mathbb{R}^{N \times N}$, where each element $w_{ij}(t)$ contains the scaled inverse travel time from node v_i to v_j at time t, such that larger values indicate shorter travel times. The geographic information for each zone is summarized in the node feature matrix $\mathbb{X} \in \mathbb{R}^{N \times d}$, which could include, e.g., location, size, or land use. An OD graph is shown in Figure 1(a), with the corresponding adjacency matrix $\mathbf{W}(t)$ illustrated in Figure 1(b).

Our goal is the following. Given a set of graphs $\{G(t) | t \in \mathcal{T}\}$, find the anomalous times $t_a \in \mathcal{T}$ corresponding to large-scale events disrupting the transportation network. We do so by training a context-augmented graph autoencoder that encodes each graph as a low-dimensional embedding

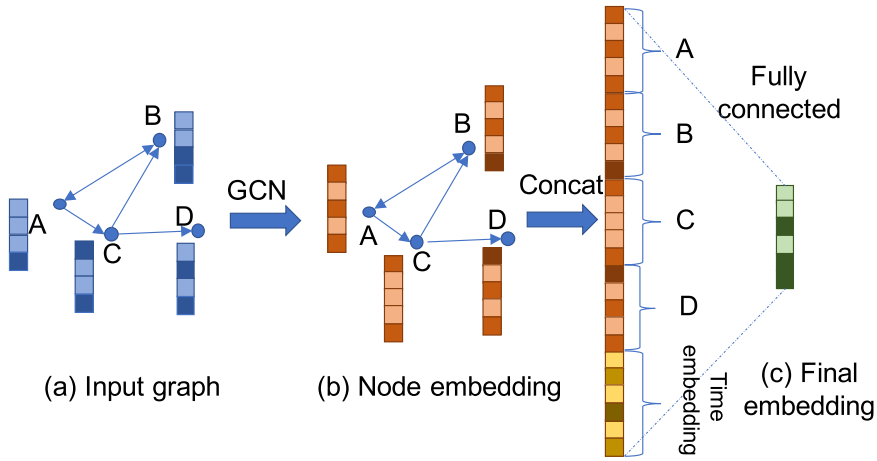


Fig. 2. Illustration of the encoder. We first apply GCN layers to aggregate the node features and edge weight information and calculate the node embedding (b). Then, we concatenate the node embedding and the time embedding, and further shrink the embedding dimension, to calculate the embedding of the graph as a whole as in panel (c).

conditioned on the time information, and decodes the embedding to minimize the average reconstruction error of the weighted adjacency matrix $\mathbf{W}(t)$. The autoencoder is trained only on normal data, so it produces high reconstruction errors when events occur.

2.2 Encoder

In an overview, the encoding process consists of three steps, as in Figure 2. We first apply GCN layers to encode the graph structure and node feature information, and learn an embedding vector for each node in the graph. We also use a distinct time embedding for each hour in the day and each day in the week to capture temporal information in the network. The node embeddings and the time embeddings are stacked in a vector, which is passed through a fully connected layer to arrive at a graph embedding of reduced dimension.

In the first step, we use a modified GraphSAGE [22] implementation of GCN to learn the node embedding, by aggregating the adjacent node information in a layer-by layer manner. The number of GraphSAGE layers stacked decides how many hops of neighborhood information is passed to each node [15]. Let $\mathcal{N}_i(t)$ denote the neighbors of node i. GraphSAGE propagates the node v_i embedding $h_i^l(t) \in \mathbb{R}^{d_l}$ in layer l of the network to the embedding a $h_i^{l+1}(t) \in \mathbb{R}^{d_{l+1}}$ as

$$\hat{h}_i^{l+1}(t) = \text{ReLU}\left(U^l \text{Concat}\left(h_i^l(t), \text{Agg}_{v_j \in \mathcal{N}_i(t)} h_j^l(t)\right)\right), \tag{1}$$

where $\hat{h}_i^{l+1}(t)$ is the non-normalized node embedding, Concat is the concatenation operator, and Agg is an aggregation operator described subsequently. The normalized node v_i embedding at layer $l+1$ is obtained by projecting onto the unit ball: $h_i^{l+1}(t) = \hat{h}_i^{l+1}(t)/\|\hat{h}_i^{l+1}(t)\|_2$.

The standard GraphSAGE [22] aggregation operators for binary undirected edges aggregate information from the embeddings of the neighbors of v_i via mean, max-pooling, or LSTM operations. To account for the time varying weighted directed edges, we apply a weighted mean aggregation function. We take $\mathcal{N}_i(t)$ as the set of nodes j for which a weighted edge e_{ji} is present at time t. Then the weighted mean aggregation operator reads

$$\text{Agg}_{v_j \in \mathcal{N}_i(t)} h_j^l(t) := \frac{\sum_{j \in \mathcal{N}_j(t)} w_{ji}(t) h_j^l(t)}{\sum_{j \in \mathcal{N}_j(t)} w_{ji}(t)}. \tag{2}$$

In the second step, we introduce context embeddings that capture the weekly and hourly periodicity of the mobility data. We define context as additional information not contained in the OD matrix that the encoder and decoder is conditioned on. In our case, we use temporal information as context. Context is implemented by concatenating two fixed length vectors: one is the embedding of the day of week information, the other is the embedding of the hour of the day. The embeddings capture our prior that traffic conditions on a specific time of day or a specific day of the week should be similar. We note that our method does not consider seasonal periodicity (i.e., repeated patterns in the summers compared to winters), or yearly patterns, although such extensions are possible to explore with longer (in temporal duration) datasets. The hour embedding is denoted as $h_{\text{hour}} \in \mathbb{R}^{d_{\text{hour}}}$, with one embedding per hour of day; similarly, the week embedding is denoted as $h_{\text{week}} \in \mathbb{R}^{d_{\text{week}}}$, with one embedding per day of the week. Given the graph at time t (e.g., 10 a.m. Tuesday Sept. 1), we use the corresponding h_{hour} (i.e., 10 a.m.) and h_{week} (i.e., Tuesday) embedding. The entries of each embedding vector are initiated from an i.i.d. normal distribution, then learned during training time.

In the last step, we combine the node and time embeddings into a single low-dimensional embedding through a fully connected layer:

$$\begin{aligned} \widetilde{h}_G(t) &= \text{Concat}\left(h_1^L(t), h_2^L(t), \ldots, h_N^L(t), h_{\text{hour}}(t), h_{\text{week}}(t)\right), \\ h_G(t) &= \text{ReLU}(U_G \widetilde{h}_G(t)), \end{aligned} \tag{3}$$

where $h_G(t) \in \mathbb{R}^{d_g}$ is the final graph embedding at time t, and $U_G \in \mathbb{R}^{d_g \times (N d_L + d_{\text{week}} + d_{\text{hour}})}$ is weight matrix.

2.3 Decoder

The decoder works in the opposite way as the encoder, as shown in Figure 3. We first calculate the node embedding from the graph embedding. Then, given each pair of node embeddings corresponding v_i and v_j at time t, we calculate the $w_{ij}(t)$ entry in the weighted adjacency matrix.

We explicitly condition the graph decoding on time by concatenating the graph embedding $h_{\mathcal{G}}(t)$ with the corresponding time embeddings. Then a fully connected layer is used to recover a vector containing all node embeddings:

$$\begin{aligned} \widetilde{h}'_G(t) &= \text{ReLU}(U'_G \text{Concat}(h_G(t), h_{\text{hour}}(t), h_{\text{week}}(t))), \\ \{h_1(t), h_2(t), \ldots, h_N(t)\} &= \text{Unstack}(\widetilde{h}'_G(t)), \end{aligned} \tag{4}$$

which is subsequently unstacked to recover the individual node embeddings $h_i(t) \in \mathbb{R}^{d_L}$. The term $U'_G \in \mathbb{R}^{N d_L \times (d + d_{\text{week}} + d_{\text{hour}})}$ is a weight matrix to be trained.

Given a pair of nodes (v_i, v_j) and the corresponding recovered node embeddings $(h_i(t), h_j(t))$, we use an edge weight predictor to recover the edge weight $w_{ij}(t)$. Note that the predominantly used GAE edge predictor $w_{ij} = \sigma(h_i^T h_j)$ [26] cannot be adopted here, because it produces symmetric results for w_{ij} and w_{ji}. Because the graph corresponding to the mobility data is directed, forcing w_{ij}, w_{ji} to be the same would introduce bias. A bilinear predictor $w_{ij} = h_i^T Q h_j$ with Q to be learned is also popular for asymmetric decoding [57], but is slightly slower and performs slightly worse in our experiments. Instead, we use a parametrized MLP as a weight predictor, by first concatenating the two node embeddings and then applying fully connected layers:

$$\begin{aligned} \widetilde{w}_{ij}(t) &= \text{ReLU}\left(U^1_{\text{dec}} \text{Concat}(h_i(t), h_j(t))\right), \\ w'_{ij}(t) &= \sigma\left(U^2_{\text{dec}} \widetilde{w}_{ij}(t)\right), \end{aligned} \tag{5}$$

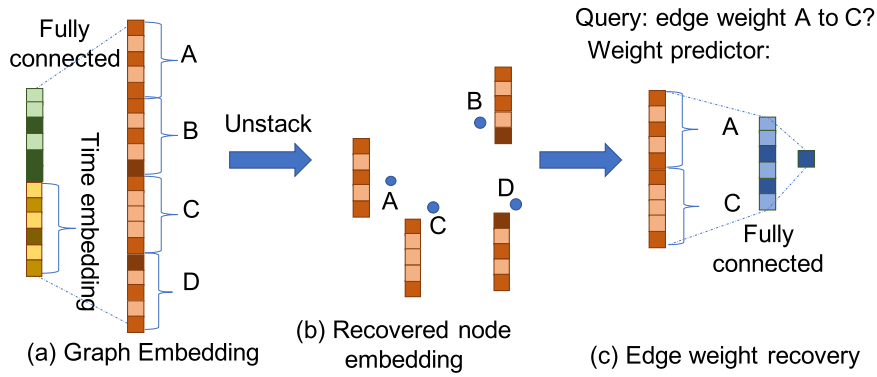


Fig. 3. Illustration of the decoder. We first calculate the node embeddings from the graph encoding conditioned on time as in panels (a) and (b). Then for each query of edge weight from one node to another, we use a weight predictor to calculate from the two corresponding node embeddings (c).

where $U^1_{\text{dec}} \in \mathbb{R}^{d_e \times 2d_L}$ and $U^2_{\text{dec}} \in \mathbb{R}^{1 \times d_e}$ are weight matrices. The final sigmoid layer σ ensures $w'_{ij}(t) \in [0, 1]$.

2.4 Loss Function

We use the ***mean-squared error*** **(MSE)** between the entries of the original and recovered weighted adjacency matrices as the loss function. For a graph instance $G(t)$ at time t, the loss function reads

$$\mathcal{L}(t) = \frac{1}{|\mathcal{E}(t)|} \sum_{e_{ij} \in \mathcal{E}(t)} (w_{ij}(t) - w'_{ij}(t))^2. \tag{6}$$

We use the Adam optimizer [25] to minimize the loss function Equation (6) for training instances in mini-batches. During training, the values of $U^1, \ldots, U^L, U_G, U'_G, U^1_{\text{dec}}, U^2_{\text{dec}}$, the 24 h_{hour}, and the 7 h_{week} time embeddings are updated. During testing, the loss function Equation (6) for each testing instance is used as its anomaly score.

2.5 Regularization

For regularization during training, we conduct edge dropout before feeding the graph data into Con-GAE, by randomly dropping edges from the edge set $\mathcal{E}(t)$ with probability $p_{\text{e_drop}}$. Edge dropout amounts to masking a portion of edges and letting Con-GAE to predict edge weights without seeing the values. This can help improve the generalization ability, and forces Con-GAE to learn the relations between nodes for correct predictions. We also apply dropout [41] with probability p_{drop} on the output of the GraphSAGE layers as well as on the time embeddings. Dropout prevents over-reliance on specific features (such as the time embedding) that leads to locally optimal reconstruction solutions.

3 EXPERIMENTS

We use three large-scale real-world transportation datasets to evaluate Con-GAE. The experiments include: (1) performance comparisons of Con-GAE with other methods for spatial and temporal anomaly detection in terms of AUC score and run time; (2) an ablation study to check the contribution of each component of Con-GAE; (3) a sensitivity study of the model hyper-parameters; (4) a robustness analysis to anomalies in the training data; and (5) a demonstration of the real-world performance of Con-GAE. The code can be found at https://github.com/yuehu9/Con-GAE.

Table 1. Dataset Statistics

Dataset	Uber Movement	NYC taxi	Chicago taxi
# graph samples	4,331	4,344	2,880
Avg. travel time (min)	5.3	14.8	13.4
Avg. # edges per graph	1,471	1,482	440

3.1 Dataset

Uber Movement [47] provides multiple publicly available mobility datasets including origin-destination travel times at an hourly basis. We use data from the first two quarters of 2019 (Jan.–June) in Nashville, TN for analysis, which is the latest data available. The Q1 data is used for training, and the Q2 data is used for testing. We note that a known large event took place during the Q2 period in Nashville, namely, the NFL draft from April 25–27. Accordingly, we separate out the three-week period containing the NFL draft, April 15–May 1 for a qualitative case study.

The ***New York City (NYC)*** *taxi dataset* [46] is an open source dataset recording pick-up and drop-off time and locations for each taxi trip in NYC. We aggregate the trip information into origin-destination travel-time matrices for taxi zones at an hourly basis. The data from Jan. 1–Mar. 31, 2019 is used for training, and Apr. 1–June 30, 2019 is used for testing.

The *Chicago taxi dataset* [10] records the taxi trips in the City of Chicago, and is processed in the same way as the NYC taxi dataset to result in origin-destination travel-time matrices at an hourly basis. The data from Jan. 1–Mar. 31, 2019 is used for training, and Apr. 1–Apr. 30, 2019 is used for testing.

Table 1 shows the statistics of the three datasets. The plots of the travel-time distributions can be found in the Supplementary Material. For all three datasets, we restrict our analysis to the top 50 zones, covering at least 75% of all data. Because of the unavailability of large-scale labeled traffic anomaly datasets, we follow the anomaly injection approach as in References [1, 6, 17], and add in synthetic events to testing sets.

3.2 Baselines

We compare our method against both traditional and state-of-the-art models for anomaly detection spanning multiple detection strategies. The baselines are: (i) a naive ***historical average*** **(HA)** approach which calculates an anomaly score as the mean-squared error between all observed OD pairs with the historical mean value at the corresponding day of week and hour of day, thereby capturing periodicity of the data: (ii) a ***robust tensor recovery and completion*** **(RTC)** [24] method that uses a low-rank tensor decomposition to capture spatial-temporal correlations and detect outliers; (iii) An autoencoder [23] using MLP for the encoder and decoder; (iv) the *Long Short-term Memory Networks-based Encoder-Decoder scheme for Anomaly Detection* (EncDec-AD) [33]; (v) a *Deep autoencoding Gaussian mixture model* (DAGMM) [70]; (vi) A RNN-based *deep structured energy-based model* REBM [62]; and two graph-based baselines: (vii) *GCN* [27]; and (viii) GraphSAGE [22]. The graph-based methods GCN and GraphSAGE have an encoder-decoder structure to encode each node in a low dimensional space, and thus can serve as an autoencoder for anomaly detection. A description of each method can be found in the Supplementary Material.

3.3 Experiment Setup

We consider two kinds of anomalies: (i) spatial anomalies, where the traffic conditions deviate from normal spatial patterns, and (ii) temporal anomalies, where traffic conditions follow a correct spatial pattern, but the conditions are not typical for the corresponding time of day.

Spatial anomalies are injected by first randomly selecting a fraction γ of time slices to pollute. For each polluted time slice, we randomly choose a fraction α of the OD pairs, and perturb the corresponding travel time by a factor drawn from an uniform distribution $\mathcal{U}(-\beta, \beta)$. Temporal anomalies are introduced by randomly selecting a fraction γ of time slices, and shifting the time associated with the data by 12 h (e.g., 8 p.m. becomes 8 a.m. and vise versa).

We choose to synthetically generate anomalies because of the lack of existing real-world labeled OD datasets that can be used to benchmark traffic anomaly detection performance, despite the now emerging OD datasets in the U.S. from major ride share providers. Namely, it is possible for cities to freely obtain historical (sparse) origin destination travel-time data, but this data has not been labeled with small or large events that may have occurred during the time period in which the data was collected. Consequently, for our work, we follow a standard anomaly injection approach. Out of different anomaly injection strategies, References [6] points out that superimposing anomalies on normal data provides more accurate testing data than simulating testing data wholly from scratch. The traffic anomaly detection work [17] uses a similar approach, that adds perturbations to the OD traffic data, and the values are drawn from a uniform distribution. More broadly, the uniform distribution is common to create gross corruptions [17, 20]. As for temporal anomalies, we choose to apply a 12 h shift, because the spatial pattern of traffic is guaranteed to be physically correct, since the traffic pattern does actually occur in the city. The time shift of 12 h is interesting, because, e.g., on a typical commute, the travel times are directional, i.e., commuters are traveling into the city in the morning, and out in the evening. The shift needs to be sufficiently large to create distinct traffic conditions; in comparison, a shift by a single hour would not typically lead to an anomaly. In short, these anomalies have real traffic conditions, but they are unusual for the given time.

In the experiments, the pollution ratio and magnitudes are set to reflect the assumption that large events are infrequent, but they pollute relatively large portions of the network. We also vary the pollution ratios and magnitudes α, β, and γ to check the anomaly detection performance under various cases. For each case, we randomly generate the test set five times and calculate the average model performance.

The ***area under the receiver operating characteristic curve*** **(AUC)** is used as the metric to compare the methods. For the deep learning methods (i.e., Con-GAE, AE, DAGMM, EncDec-AD, REBM, GCN, and GraphSAGE), the output of a given sample is an anomaly score. Using the anomaly scores of the test samples, the AUC is computed. For RTC, the output includes a sparse tensor containing anomalies. A anomaly score is calculated using the l_2 norm of the vector corresponding to each sample.

3.4 Model Configurations

The model settings for Con-GAE are as follows. The node feature matrix $X \in \mathbb{R}^{N \times d}$ is constructed with $d = 4$ corresponding to the minimum and maximum latitude and longitude extents of the zone corresponding to the node. The encoder uses $L = 2$ GCN layers to learn the node-level embedding. The model is trained for 150 epochs with a batch size of 10. Of the training set, 10% is kept out as a validation set for hyper-parameter tuning. Other settings of Con-GAE for different datasets are as follows.

For the Uber Movement data, the dimension of the two node embeddings are $h_i^1(t) \in \mathbb{R}^{100}$ and $h_i^2(t) \in \mathbb{R}^{100}$, respectively. The dimension of the hour and week embeddings are $d_{\text{week}} = d_{\text{hour}} = 200$, and the graph embedding dimension is $d_g = 100$. The dropout rates $p_{\text{e_drop}}$ and p_{drop} are both set at 0.2. The initial learning rate of the Adam optimizer is 1×10^{-5}, and we decay the learning rate by 0.5 every 50 epochs.

Table 2. AUC Score for Spatial and Temporal Anomaly Detection of Different Datasets

Anomaly type	Spatial anomaly			Temporal anomaly		
Dataset	Uber	NYC	Chicago	Uber	NYC	Chicago
HA	0.694	0.498	0.570	0.822	0.677	0.658
RTC	0.856	0.795	0.601	0.559	0.247	0.562
AE	0.772	0.671	0.727	0.548	0.592	0.460
EncDec-AD	0.523	0.782	0.715	0.481	0.659	0.671
REBM	0.762	0.759	0.739	0.484	0.805	0.482
DAGMM	0.514	0.603	0.669	0.484	0.397	0.414
GraphSAGE	0.679	0.741	0.674	0.499	0.464	0.426
GCN	0.659	0.593	0.765	0.494	0.416	0.421
Con-GAE	**0.895**	**0.837**	**0.912**	**0.868**	**0.895**	**0.753**

For the NYC taxi data, the dimension of the two node embeddings are $h_i^1(t) \in \mathbb{R}^{150}$ and $h_i^2(t) \in \mathbb{R}^{50}$, respectively. The dimension of the hour and week embeddings are $d_{\text{week}} = d_{\text{hour}} = 100$, and the graph embedding dimension is $d_g = 50$. The dropout rates $p_{\text{e_drop}}$ and p_{drop} are both set at 0.2. The initial learning rate is 1×10^{-3}, and we decay the learning rate by 0.5 every 20 epochs.

For the Chicago taxi data, the dimension of the two node embeddings are $h_i^1(t) \in \mathbb{R}^{300}$ and $h_i^2(t) \in \mathbb{R}^{25}$, respectively. The dimension of the hour and week embeddings are $d_{\text{week}} = d_{\text{hour}} = 200$, and the graph embedding dimension is $d_g = 25$. The dropout rates $p_{\text{e_drop}}$ and p_{drop} are both set at 0.1. The initial learning rate is 1×10^{-3}, and we decay the learning rate by 0.5 every 20 epochs.

The AE, DAGMM, EncDec-AD, and REBM are implemented based on the code in Reference [19]. the respective hyper-parameters for each model are tuned to archive the best performance. GCN and GraphSAGE have the same encoding node dimension as our model. Detailed configurations are available in the Supplementary Materials.

3.5 Result and Analysis

3.5.1 Model Comparison. First, we compare Con-GAE with the baseline methods for both spatial and temporal anomaly detection, for different datasets. We fix the fraction of the time slices chosen to be polluted $\gamma = 10\%$, pollution magnitude $\alpha = 50\%$ and $\beta = 10\%$. The results are shown in Table 2. We can see that Con-GAE constantly outperforms the other methods, with an improvement in the AUC score as large as 0.15 over the second best method. For spatial anomalies, Con-GAE is the only method to achieve an AUC close to or above 0.9 for the Uber data and the Chicago data, and the only method above 0.8 for the NYC data. For temporal anomalies, most other methods have an AUC score around 0.5 except HA, REBM, and EncDec-AD, meaning they are not well suited for detecting temporal anomalies, while Con-GAE achieves an AUC score of 0.75 or higher. These experiments highlight that when dealing with large-scale mobility data, taking the graph structure into account can substantially boost the performance. Meanwhile, directly applying the GCN and GraphSAGE model results in poor performance, showing the importance of designing a hierarchical structure for a graph-scale embedding beyond the individual node embeddings when detecting network-wide anomalies. The temporal experiments show that for data with periodicity, it is more effective to consider the long-term periodicity as in Con-GAE and HA, than to consider the short-term dependencies as in LSTM-ED and REBM, or no temporal dependencies as in DAGMM, AE, GCN, and GraphSAGE.

Next, we compare the anomaly detection methods under different anomaly rates. We vary the fraction γ of the time slices chosen to be polluted, fixing $\alpha = 50\%$ and $\beta = 10\%$. The result is shown

Table 3. AUC Score for Spatial and Temporal Anomaly Detection Under Different Anomaly Rates

Anomaly type	Spatial anomaly			Temporal anomaly		
Anomaly rate γ	5%	10%	20%	5%	10%	20%
HA	0.690	0.694	0.705	0.838	0.822	0.816
RTC	0.896	0.856	0.865	0.671	0.559	0.548
AE	0.770	0.772	0.757	0.544	0.548	0.514
EncDec-AD	0.576	0.523	0.505	0.499	0.481	0.517
REBM	0.740	0.762	0.737	0.456	0.484	0.501
DAGMM	0.481	0.514	0.538	0.460	0.484	0.479
GraphSAGE	0.730	0.679	0.704	0.569	0.499	0.510
GCN	0.717	0.659	0.689	0.548	0.494	0.514
Con-GAE	**0.910**	**0.895**	**0.909**	**0.897**	**0.868**	**0.865**

We vary the fraction γ of the time slices chosen to be polluted.

Table 4. AUC Score for Spatial Anomaly Detection Under Different Magnitudes of Anomalies

Spatial anomaly rate α	25%			50%		
Anomaly magnitude β	5%	10%	20%	5%	10%	20%
HA	0.500	0.588	0.812	0.531	0.694	0.942
RTC	**0.743**	0.798	0.921	**0.765**	0.856	0.973
AE	0.570	0.672	0.866	0.606	0.772	0.973
EncDec-AD	0.492	0.505	0.541	0.498	0.523	0.606
REBM	0.607	0.679	0.867	0.634	0.762	0.968
DAGMM	0.509	0.515	0.483	0.507	0.514	0.501
GraphSAGE	0.602	0.638	0.739	0.615	0.679	0.841
GCN	0.594	0.622	0.706	0.606	0.659	0.798
Con-GAE	0.681	**0.800**	**0.963**	0.729	**0.895**	**1.000**

Results are shown under different α deciding the OD pairs polluted, and different β corresponding to the perturbation magnitude.

in Table 3, where Uber Movement data is used. We can see that the advantage of our method holds under different anomaly rates.

We also investigate the influence of the anomaly magnitude. We focus on spatial anomaly detection, and vary the fraction α of OD pairs polluted and the uniform distribution $\mathcal{U}(-\beta, \beta)$ controlling the magnitude of the pollution. We fix the time slices polluted at $\gamma = 10\%$. The result is shown in Table 4, where Uber Movement data is used. No method obtains the best performance over all scenarios. All methods have higher AUC score with larger α and β, and perform less well when the pollution is more nuanced. Con-GAE has significant advantage over the other methods for $\beta = 10\%$ or larger. When β is lower than 10%, our method is outperformed by the linear RTC method by a small margin of around 0.05. Since our goal is to detect large events causing a significant disturbance to road networks, we conclude that Con-GAE is the most competitive method overall.

Last, we examine the efficiency of the models. We compare the run time for a single sample, i.e., a time slice of a single hour. We average the run time over all testing data, by computing one testing sample at a time. The result is summarized in Table 5. We find that the graph-based

Table 5. Run Time for a Single Sample, i.e., a Time Slice of a Single Hour

Model	RTC	AE	EncDec-AD	REBM	DAGMM	GraphSAGE	GCN	Con-GAE
Run time (s)	244.7	0.0047	0.1285	0.0413	0.0305	0.0012	0.0018	0.0022

Table 6. Ablation Study

Anomaly type	Spatial anomaly			Temporal anomaly		
Anomaly rate γ	5%	10%	20%	5%	10%	20%
Con-GAE	**0.910**	**0.895**	**0.909**	**0.897**	**0.868**	**0.865**
Con-GAE-sp	0.877	0.835	0.855	0.552	0.493	0.507
Con-GAE-t	0.820	0.794	0.809	0.798	0.748	0.735
Con-GAE-fc	0.768	0.784	0.741	0.714	0.720	0.736
Con-GAE-NonWeightedEnc	0.830	0.799	0.820	0.810	0.773	0.763

methods have the smallest run time, followed by deep neural networks without a graph structure. This is because the graph-based methods do not need to consider all N^2 edges at each step. Instead, by utilizing node embedding techniques and computing only the sparsely connected edges, the computational complexity is on the order of $N + E$, where N is the number of nodes and E is the number of edges. With 50 nodes, we can already see the advantage of graph-based methods. The robust tensor decomposition method has the longest run time, since at each step it requires computing a singular value decomposition over all historical data.

3.5.2 Ablation Study. We compare the several variants of Con-GAE, to see how each component of Con-GAE helps with anomaly detection. We consider the following variants: (i) *Con-GAE-sp*: We remove all temporal information of the autoencoder, and only use the spatial graph data for training and detection. (ii) *Con-GAE-t*: We remove the GCN layers, and only use the temporal information for training and detection. (iii) *Con-GAE-fc*: We replace the GCN layers in the autoencoder with fully connected layers, while temporal information is retained. (iv) *Con-GAE-NonWeightedEnc*: We use the standard GraphSAGE implementation Equation (1) with a mean aggregator in encoder, instead of the modified weighted mean aggregator.

The result is shown in Table 6, where Uber Movement data is used. We can see that the modified GCN layers play a crucial role in anomaly detection. The AUC score is around 0.1 lower if we remove the GCN layers (Con-GAE-t), replace GCN layers with fully connected layers (Con-GAE-fc), or replace GCN with standard GraphSAGE layers (Con-GAE-NonWeightedEnc). We observe that temporal information provides marginal benefit for spatial anomaly detection, but is crucial for temporal anomaly detection. Without temporal information, it is impossible for Con-GAE-sp to detect temporal anomalies, with an AUC score constantly around 0.5.

3.5.3 Comparison on Graph Convolutional Layers. With the rapid development of graph neural networks, new mechanisms of graph convolution have been designed. These graph convolutional layers can easily be adapted into our Con-GAE framework in place of the weighted GraphSAGE. In this section, we compare the performance of weighted GraphSAGE with other graph convolutional layers that incorporate edge weights when aggregating neighborhood information. We include: (i) TransformerConv [40] that uses attention mechanism, (ii) ARMAConv [4], which uses a recursive formulation inspired by the auto-regressive moving average (ARMA) filter, (iii) TAGConv [14], which is a topology adaptive graph convolutional network, and (iv) SGCConv [54], which removes nonlinearities and collapses weight matrices successively to reduce complexity of the GCN.

Table 7. Performance of other Graph Convolutional Layers Besides Weighted GraphSAGE

Anomaly type	Spatial anomaly			Temporal anomaly		
Anomaly rate γ	5%	10%	20%	5%	10%	20%
Weighted GraphSAGE	0.910	0.895	0.909	**0.897**	0.868	**0.865**
TransformerConv	**0.949**	**0.940**	**0.950**	0.750	0.711	0.692
ARMAConv	0.917	0.902	0.921	0.887	**0.869**	0.864
TAGConv	0.911	0.896	0.917	0.863	0.852	0.845
SGCConv	0.733	0.712	0.742	0.558	0.516	0.502

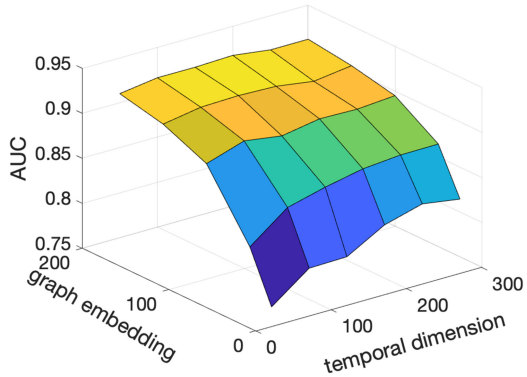


Fig. 4. Sensitivity analysis of Con-GAE. AUC score of Con-GAE with varying dimension of graph and time embedding is plotted.

The parameters for each one are tuned for best performance, and are recorded in the Supplementary Materials.

The result is shown in Table 7, where the Uber movement dataset is used. We can see that all graph convolutional layers reach comparable performance except SGCConv. TransformerConv has the best performance in detecting spatial anomalies, but performs less well on temporal anomaly detection. In comparison, weighted GraphSAGE performs the best on temporal anomalies, and also adequately well on spatial anomalies with only 0.04 less in the AUC score compared to the best method. Thus, we choose weighted GraphSAGE in our Con-GAE model implementation.

3.5.4 Sensitivity Analysis. Next, we explore how the performance of Con-GAE changes as a function of the graph embedding dimension, the hour and week embedding dimension, and the model depth. We fix $\alpha = 50\%$, $\beta = 10\%$, and $\gamma = 10\%$. Uber Movement data is used.

First, we explore the sensitivity to spatial and temporal encoding dimensions. We vary the graph embedding dimension d_g between 25 and 200, and vary the week and hour embedding dimension between 50 and 300. The dimension of node embeddings h_i^1 and h_i^2 is set the same as graph embedding dimension d_g. We fix the other parameters to be the same those described in Model Configurations. The result is shown in Figure 4. We can see that as the graph dimension and temporal dimension increases, the AUC score first increases rapidly, then stays steady with slight increase.

Next, we investigate the sensitivity to model depth. We vary the number of GCN layers from 1 to 6, fixing $h_i^l(t) \in \mathbb{R}^{100}$ for all layers. The result is shown in Table 8. We can see that when the number of layers is below 4, the model has about the same performance. But when the number of layers is above 4, the performance degrades with depth.

Table 8. Sensitivity to Model Depth

# GCN layers	1	2	3	4	5	6
AUC score	0.887	0.895	0.895	0.850	0.853	0.785

Table 9. Robustness to Pollution of Anomalies in Training Data

Anomaly type	Spatial anomaly			Temporal anomaly		
Anomaly rate γ	5%	10%	20%	5%	10%	20%
Original training	0.9098	0.8954	0.9089	0.8973	0.8679	0.8649
Spatially polluted training	0.9096	0.8951	0.9089	0.8979	0.8684	0.8654
Temporally polluted training	0.9034	0.8888	0.9035	0.8570	0.8244	0.8221

3.5.5 Robustness Analysis. Last, we study the tolerance of Con-GAE to a small amount of gross pollution in the training data. This is to understand the potential pitfalls that could occur if the training data is not clean and does in fact contain anomalies. We manually corrupt 5% of the training data spatially or temporally, following the same procedure as is used in the synthetic generation of anomalies in the test data. When injecting spatial anomalies, we set the pollution strength $\alpha = 50\%, \beta = 10\%$. We report the testing results under different anomaly rates in the test scenarios given a fixed 5% anomaly rate injected in the training data. For each case, we randomly generate the training set five times and calculate the average model performance. The result is shown in Table 9, where we find that with spatially polluted training data, the model performance stays the same. With temporally polluted training data, the AUC score downgrades slightly by at most 0.04. Thus, we find the method can tolerate anomalies in the training data without noticeably impacting performance.

3.6 Visualization of Real-world Traffic Anomaly

In this section, we demonstrate the real-world application of Con-GAE. We use the test set 4/27–5/01 of 2019 Q2, which is not seen during training and is known to contain a large event (the NFL draft). The NFL draft brought several hundred thousand visitors to the city, and nightly events resulted in roadway closures in the downtown Nashville area.

Figure 5 shows the anomaly score (the reconstruction loss) for each hour of the studied period. The days under which the major activities of the draft take place are highlighted in red on the y-axis. We can see that the highest anomaly scores occur during the draft, specifically 4/26 19:00 until midnight, as well as 4/27 15:00–18:00. These are the times when the draft, post draft, and post concert and fireworks take place [36].

Figure 6 plots the OD travel-time data during two periods with high anomaly scores. For clarity, only a subset of travel-time pairs are plotted, and only deviations in excess of 5 min from the historical average are shown. Figure 6(a) shows that the travel times from the region containing the draft to other parts of the city is significantly longer than usual. The third largest anomaly period occurs in the consecutive days of 4/21–4/24 at 20:00. This matches the record from the *Tennessee Department of Transportation* [44] that there are alternating nightly maintenance road closures for the major freeway connecting downtown to the airport, resulting in substantially longer travel times (Figure 6(b)).

4 RELATED WORK

In this section, we review graph network embedding and autoencoder-based anomaly detection techniques that inspire Con-GAE. We also review the closely related topic of spatio-temporal graphs in transportation.

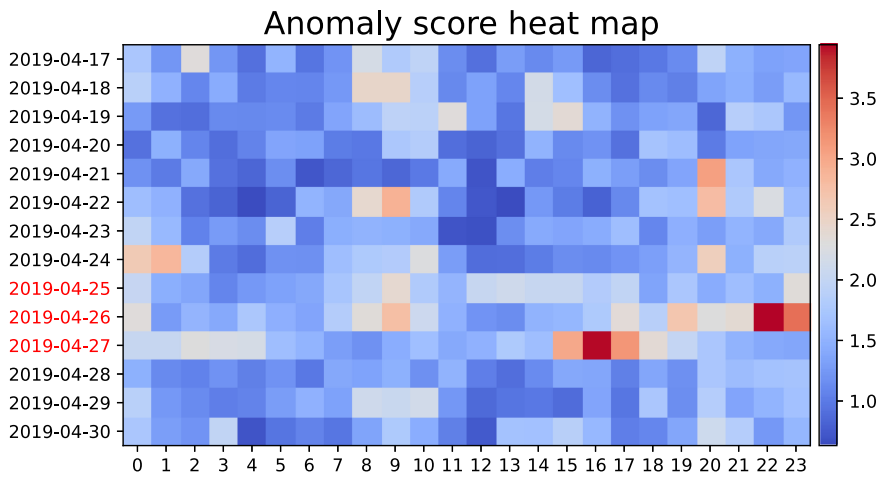


Fig. 5. Anomaly scores for each hour. The NFL draft days are marked red. The score denotes the reconstruction error Equation (6).

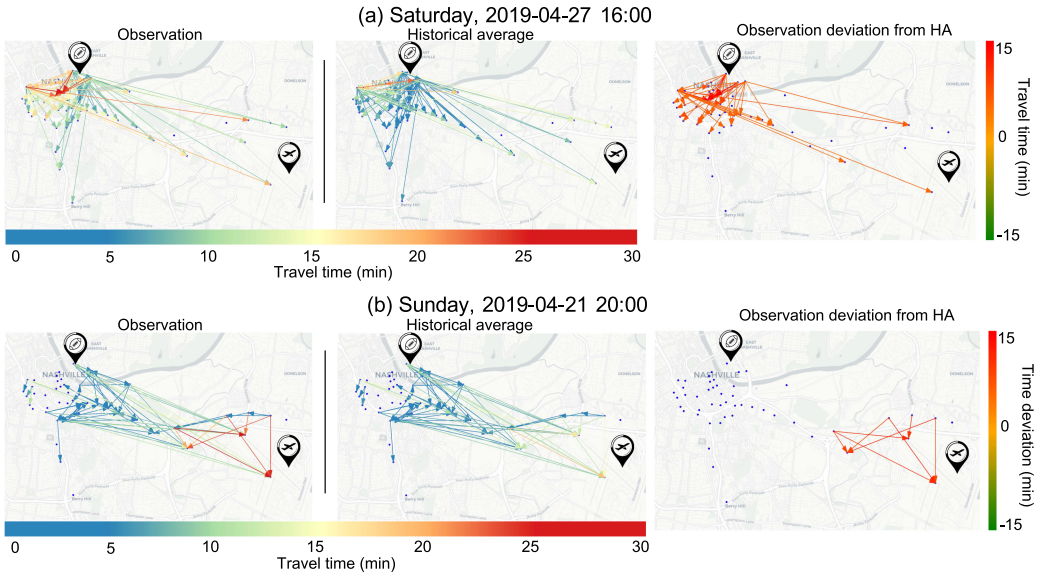


Fig. 6. Traffic conditions for the detected events. The OD pair travel time and comparison to historical conditions are plotted.

4.1 Graph Network Embedding

In recent years, there has been rapid development in graph embedding-based learning algorithms. In particular, GCN has gained popularity because of its scalability and good performance [15]. Some benchmark variations of GCN include [22, 27, 48], which have become the building block of many complex graph network models [55]. For comprehensive introductions and surveys of graph neural networks, one can refer to References [55, 67]. Our work focuses on weighted directed graphs appropriate for mobility datasets, compared with binary undirected graphs in the original work of [22, 27].

A ***graph autoencoder*** **(GAE)** is an unsupervised learning approach that encodes the graph in a latent space, and reconstructs the graph structure from the encoding [55]. Since the introduction of GAE in References [26, 45], it has been widely used, for example in link prediction [39, 53, 57], matrix completion [3], and graph clustering [49]. While existing GAE approaches such as [26] learn node embeddings during encoding, our model further compresses the node embeddings of all nodes in a graph to a single graph embedding, and conditions it on temporal information to add additional context important for interpreting mobility data.

Given the intrinsic graph structure of transportation networks, recent research has demonstrated advantages of applying graph embedding techniques on mobility data, for forecasting [12, 59], passenger demand prediction [51], dockless public bicycle station recommendation [8], travel-time estimation [50], and human trajectory prediction [35]. While graph embedding for traffic anomaly detection is less studied, our present work shows the ability of graph embeddings to learn patterns in OD mobility data for anomaly detection.

4.2 Autoencoder-based Anomaly Detection

Deep autoencoder-based methods have been widely used for anomaly detection [5, 11, 33, 38, 56, 66]. The encoder and decoder can have various structures based on different input data types. Besides the widely used MLP-based encoder and decoders [5, 38, 56], ***convolutional neural networks*** **(CNN)** are applied on grid data [11], and ***recurrent neural networks*** **(RNN)** are applied on time series data [33, 42]. Recently, GCN-based autoencoders have been used for anomaly detection on graph data [13, 16, 28, 29, 31, 69], with an important emphasis node-level anomalies. In comparison, our work aims at graph-level anomaly detection, since each graph corresponds to the network wide traffic condition at a specific time instant that may be disturbed by a large-scale event.

4.3 Spatio-temporal Graph in Transportation

The majority of works leveraging spatio-temporal graphs in transportation have focused on traffic forecasting. A representative strategy is to combine GNN and recurrent neural networks so that both spatial and temporal patterns can be captured [32, 60]. Attention mechanisms on graphs are employed to capture the variance in spatial-temporal dependencies [30, 52, 63]. Dynamic graphs are used to match the constantly changing nature of traffic [37, 64]. Existing works on spatio-temporal graphs mainly focus on modeling the short-term dependencies of consecutive graph instances. In comparison, we focus on exploiting the temporal periodicity of traffic graphs over longer time horizons for our purpose of anomaly detection.

5 CONCLUSION

In this work, we address the problem of detecting network-wide events in large origin-destination mobility datasets. We formulate the problem of detecting these events as a problem of detecting anomalies in a set of time-dependent directed graphs containing mobility data on the network. We introduced Con-GAE, which is an autoencoder-based anomaly detector that uses GCN layers and temporal embeddings to determine anomalies. Extensive experiments on three real-world mobility datasets show that Con-GAE outperforms several state-of-art anomaly detection methods in both spatial and temporal anomaly detection. In future work, we are interested to explore scalability of Con-GAE to networks with significantly more nodes, and for anomaly detection on other network datasets.

APPENDIX

A SUPPLEMENTARY MATERIAL

A.1 Experiment Data Preparation and Simulation

In this section, we detail the data reprocessing and simulation steps. We also show the data distribution of travel times for the three datasets used in the article.

As described in the main text, the dataset is split into training and testing sets. The training set is assumed to have no major events (anomalies). To compare methods in a controlled setting, we create a test set with synthetic labeled anomalies, which are injected into cleaned traffic conditions. We synthetically generate the test set from real data due to the unavailability of city scale traffic datasets pre-labeled with anomalies.

To generate cleaned traffic conditions in the test set, which are later polluted with anomalies, we assume the travel time of each OD-pair in each hour of day in each day of week follows a Gaussian distribution. We calculate the mean and variance of the OD pair travel time of corresponding hours from the raw test data from the second quarter of 2019. We then resample the travel times based on the calculated mean and variance to arrive at a complete clean test dataset. Next, we inject spatial and temporal anomalies, respectively, as described in the main text to arrive at the labeled test set with anomalies.

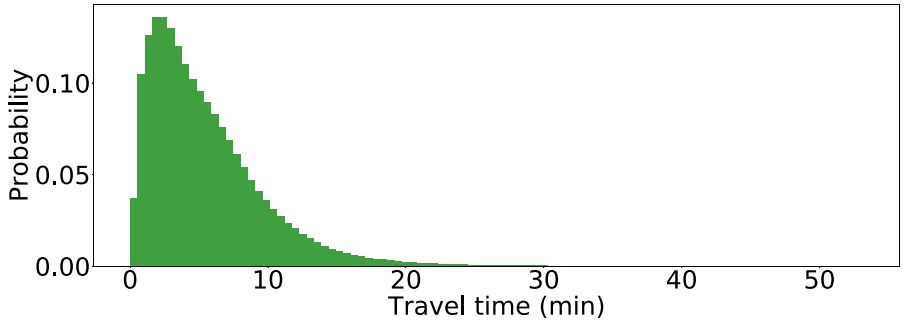
In the Uber Movement dataset corresponding to Nashville, TN, the region is divided into 2219 zones. However, many of the OD pairs never record travel times. To learn as much useful information as possible and ensure learning efficiency, we select the top 50 most connected zones, so that the selected zone has one or more trip to at least one third of the total 2,219 zones. This results in 30% missing data rate in the training set, and a 41% missing data rate in the test set. The minimum and maximum latitude and longitude extents of each zone are used as the node features, which are scaled to 0–1 before feeding into Con-GAE.

The data distribution of travel times for the three datasets used in the article are shown in Figure 7. We can see that Uber Movement dataset contains trips of shorter duration than NYC or Chicago taxi data. Meanwhile, Uber data and Chicago taxi data has more concentrated travel-time distribution than NYC taxi data.

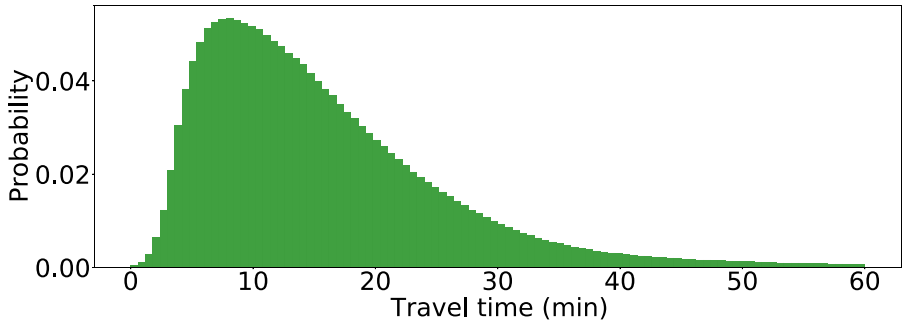
A.2 Baseline Methods

In this section, we give a brief introduction to each method and document the model configurations.

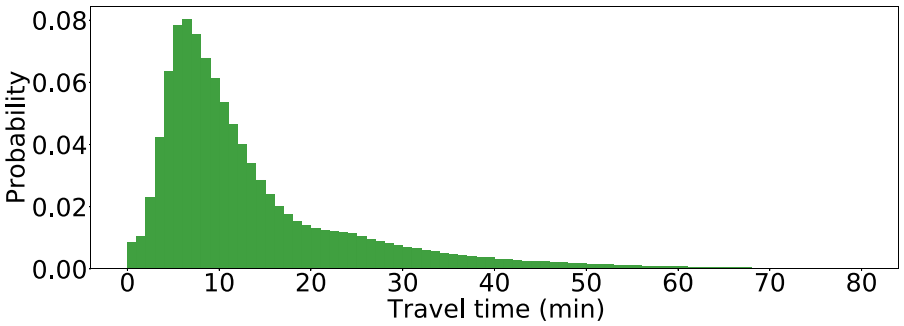
- RTC [24]. Robust tensor recovery and completion for group anomaly detection (RTC) is a low-rank tensor decomposition based method that exploits spatio-temporal correlation. This a non-deep learning baseline method. RTC has one hyper parameter λ, which controls the tradeoff between the rank of the normal tensor and the size and magnitude of the outlier tensor. We set $\lambda = 0.227$ for Uber data and Chicago data, and set $\lambda = 0.390$ for NYC data. The value is chosen empirically following the settings described in the article.
- AE [23]. Autoencoder (AE) is a widely used neural network model for anomaly detection. It uses a fully connected neural network to build an encoder and a decoder to compress and reconstruct the data. It uses the reconstruction error as the anomaly score. We set the hidden size (i.e., the dimension of the low dimensional encoded state) of AE as the same setting used in Con-GAE. Namely, the hidden size is 100 for Uber data, 50 for NYC, and 25 for Chicago data. The learning rate is 0.01 for Uber, and 0.0005 for NYC and Chicago. The number of epochs is 150 for Uber and 30 for NYC and Chicago. The values were determined to maximize performance using grid search. The sequential length is set as 1, i.e., each



(a) Travel time PDF of Uber Movement data



(b) Travel time PDF of NYC taxi data



(c) Travel time PDF of Chicago taxi data

Fig. 7. Travel-time distribution for three datasets.

input sample contains the traffic conditions of one time step. All other parameters are set as the default values, with random seed set at 0.

- EncDec-AD [33]. EncDec-AD is an LSTM-based encoder-decoder model for time series anomaly detection. EncDec-AD learns the normal pattern of a multi-dimensional time series and detects anomalous times that deviate from the normal pattern via the reconstruction error. We set the hidden size (i.e., the dimension of the low dimensional encoded state) of EncDec-AD via grid search. Namely, the hidden size is 150 for Uber data, 50 for NYC, and 25 for Chicago data. The learning rate and number of epochs are 0.0001 and 120 for Uber data, 0.0005 and 30 for NYC, and 0.001 and 70 for Chicago data. EncDec-AD takes time series windows as input, and we set the sequential length at 72 for Uber and NYC data, and 24 for

Chicago data, which are also determined via grid search. All other parameters are set as the default values, with random seed set at 0.

- DAGMM [70]. ***Deep autoencoding Gaussian mixture model* (DAGMM)** conducts unsupervised anomaly detection by finding low-dimensional data representations via a deep autoencoder, and modeling the data distribution in the low-dimensional space via a Gaussian mixture model. Instead of setting the hidden size of DAGMM the same as Con-GAE, a grid search revealed the best performance is achieved at a hidden size of 50 for Uber data and NYC data, and 5 for Chicago data. While the other methods (Con-GAE, AE, EncDec-AD) are reconstruction-based, DAGMM relies on clustering in a low-dimensional space. An encoding dimension too low might not capture the original information, while high-dimensional clustering can encounter a curse of dimensionality. The learning rate and number of epochs are 0.001 and 20 for Uber data, 0.0005 and 50 for NYC data, and 0.0001 and 20 for Chicago data, which yields the best result after parameter grid search. The sequential length is set as 1, i.e., each input sample contains the traffic conditions of one time step. The rest parameters are set at the default values, with the random seed at 0.
- REBM [62]. REBM is the RNN-based version of *deep structured* ***energy-based models* (EBM)** for time series anomaly detection. REBM models the data distribution with neural networks, and detects the anomaly by the energy (negative log probability) and reconstruction error. We set the hidden size (i.e., the dimension of the low dimensional encoded state) of AE as the same setting used in Con-GAE. Namely, the hidden size is 150 for Uber data, 50 for NYC, and 25 for Chicago data. The learning rate and number of epochs are 0.0005 and 50 for Uber data, 0.0001 and 100 for NYC, and 0.001 and 50 for Chicago data. All other parameters are set as the default values, with random seed set at 0.
- GCN [27]. As a baseline formulation of Graph neural network, GCN updates node features by averaging over the features of neighbor nodes. We use the final node feature matrix of all nodes as the encoding for the traffic graph, and use the same decoder as our Con-GAE model to decode the edge weights from the node encoding. The reconstruction error across all edges is used as anomaly score. For GCN, we use the same structure as Con-GAEmodel. Namely, we use two layers of GCN, the node embeddings are set at 300 and 150 for Uber data; 150 and 50 for NYC data, and 300 and 25 for Chicago data. The dropout rate is set at 0.2. The learning rate is set at 0.0001 for Uber data, 0.001 for NYC data, and 0.01 for Chicago data.
- GraphSAGE [22] improves from GCN by explicitly concatenating the feature of the node itself after aggregating the neighboring node features. Its encoding-decoding structure is the same as our GCN comparison method. It is akin to our Con-GAE model without the aggregating layer from node embedding to graph embedding and without the time embedding. For GraphSAGE, we use the same structure as Con-GAEmodel. Namely, we use two layers of GCN, the node embeddings for the two layers are set at 300 and 150 for Uber data; 150 and 50 for NYC data, and 300 and 25 for Chicago data. The dropout rate is set at 0.2. The learning rate is set at 0.0001 for Uber data, 0.0005 for NYC data, and 0.05 for Chicago data.

For the methods that do not directly accommodate missing data, namely, HA, AE, DAGMM, EncDec-AD, and REBM, we impute the missing data via temporal interpolation (i.e., via linear interpolation when values are present to interpolate, or via a mean value when interpolation cannot be performed).

A.3 Model Configurations for Comparison Graph Convectional Layers

In this section, we record the parameter settings for the models in Table 7, where we compare different graph convolutional layers.

For all methods, the node features and training-testing split is the same. The encoder uses $L = 2$ graph convolutional layers to learn the node-level embedding. The dropout rates $p_{\text{e_drop}}$ and p_{drop} are both set at 0.2.

For TransformerConv, the dimension of the two node embeddings are $h_i^1(t) \in \mathbb{R}^{100}$ and $h_i^2(t) \in \mathbb{R}^{100}$, respectively. The dimension of the hour and week embeddings are $d_{\text{week}} = d_{\text{hour}} = 50$, and the graph embedding dimension is $d_g = 100$. The number of multi-head-attention head is 10. The initial learning rate of the Adam optimizer is 1×10^{-4}, and we decay the learning rate by 0.5 every 100 epochs.

For ARMAConv, the dimension of the two node embeddings are $h_i^1(t) \in \mathbb{R}^{300}$ and $h_i^2(t) \in \mathbb{R}^{300}$, respectively. The dimension of the hour and week embeddings are $d_{\text{week}} = d_{\text{hour}} = 50$, and the graph embedding dimension is $d_g = 100$. The initial learning rate of the Adam optimizer is 5×10^{-5}, and we decay the learning rate by 0.5 every 100 epochs.

For TAGConv, the dimension of the two node embeddings are $h_i^1(t) \in \mathbb{R}^{300}$ and $h_i^2(t) \in \mathbb{R}^{300}$, respectively. The dimension of the hour and week embeddings are $d_{\text{week}} = d_{\text{hour}} = 50$, and the graph embedding dimension is $d_g = 150$. The initial learning rate of the Adam optimizer is 5×10^{-5}, and we decay the learning rate by 0.5 every 100 epochs.

For SGCConv, the dimension of the two node embeddings are $h_i^1(t) \in \mathbb{R}^{300}$ and $h_i^2(t) \in \mathbb{R}^{300}$, respectively. The dimension of the hour and week embeddings are $d_{\text{week}} = d_{\text{hour}} = 50$, and the graph embedding dimension is $d_g = 150$. The initial learning rate of the Adam optimizer is 1×10^{-4}, and we decay the learning rate by 0.5 every 100 epochs.

A.4 Reproducibility

In this section, we specify the computing hardware and software used for experiments, as well as the data availability.

The GPU used for the experiments is a GTX 1080 (12 GB) operated on Ubuntu 18.04.4 LTS. We use the python package Pytorch Geometric [18] version 1.7 to implement the graph convolutional layers.

The Uber Movement time series data is from Nashville, TN, which is not yet publicly available on the Uber Movement portal [47]. Uber Movement data is available under a Creative Commons, Attribution Non-Commercial license.[1]

REFERENCES

[1] Leman Akoglu, Hanghang Tong, and Danai Koutra. 2015. Graph based anomaly detection and description: A survey. *Data Min. Knowl. Discov.* 29, 3 (2015), 626–688.

[2] Julien Audibert, Pietro Michiardi, Frédéric Guyard, Sébastien Marti, and Maria A. Zuluaga. 2020. USAD: UnSupervised anomaly detection on multivariate time series. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 3395–3404.

[3] Rianne van den Berg, Thomas N. Kipf, and Max Welling. 2018. Graph convolutional matrix completion. In *Proceedings of the International Conference on Knowledge Discovery and Data Mining (KDD'18)*.

[4] Filippo Maria Bianchi, Daniele Grattarola, Lorenzo Livi, and Cesare Alippi. 2022. Graph Neural Networks With Convolutional Arma Filters. In *IEEE Trans. Pattern Anal. Mach. Intell.* 44, 7 (2022), 3496–3507. DOI: 10.1109/TPAMI.2021.3054830

[5] Andrea Borghesi, Andrea Bartolini, Michele Lombardi, Michela Milano, and Luca Benini. 2019. Anomaly detection using autoencoders in high performance computing systems. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 9428–9433.

[6] David L. Buckeridge, Howard Burkom, Murray Campbell, William R. Hogan, Andrew W. Moore, et al. 2005. Algorithms for rapid outbreak detection: A research synthesis. *J. Biomed. Inform.* 38, 2 (2005), 99–113.

[1] https://movement.uber.com/faqs?lang=en-US.

[7] Saikiran Bulusu, Bhavya Kailkhura, Bo Li, Pramod K. Varshney, and Dawn Song. 2020. Anomalous example detection in deep learning: A survey. *IEEE Access* 8 (2020), 132330–132347.
[8] Jianguo Chen, Kenli Li, Keqin Li, Philip S. Yu, and Zeng Zeng. 2021. Dynamic planning of bicycle stations in dockless public bicycle-sharing system using gated graph neural network. *ACM Trans. Intell. Syst. Technol.* 12, 2 (2021), 1–22.
[9] Chicago. 2020. Divvy Trips. Retrieved Nov. 2020 from https://data.cityofchicago.org/Transportation/Divvy-Trips/fg6s-gzvg.
[10] Chicago. 2020. Taxi Trips. Retrieved Nov. 2020 from https://data.cityofchicago.org/Transportation/Taxi-Trips/wrvz-psew.
[11] Yong Shean Chong and Yong Haur Tay. 2017. Abnormal event detection in videos using spatiotemporal autoencoder. In *Proceedings of the International Symposium on Neural Networks*. Springer, 189–196.
[12] Zulong Diao, Xin Wang, Dafang Zhang, Yingru Liu, Kun Xie, and Shaoyao He. 2019. Dynamic spatial-temporal graph convolutional neural networks for traffic forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 890–897.
[13] Kaize Ding, Jundong Li, Rohit Bhanushali, and Huan Liu. 2019. Deep anomaly detection on attributed networks. In *Proceedings of the SIAM International Conference on Data Mining*. SIAM, 594–602.
[14] Jian Du, Shanghang Zhang, Guanhang Wu, José M. F. Moura, and Soummya Kar. 2017. Topology adaptive graph convolutional networks. Retrieved from https://arXiv:1710.10370.
[15] Vijay Prakash Dwivedi, Chaitanya K. Joshi, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. 2020. Benchmarking graph neural networks. Retrieved from https://arXiv:2003.00982.
[16] Haoyi Fan, Fengbin Zhang, and Zuoyong Li. 2020. AnomalyDAE: Dual autoencoder for anomaly detection on attributed networks. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP'20)*. IEEE, 5685–5689.
[17] Hadi Fanaee-T. and Joao Gama. 2016. Event detection from traffic tensors: A hybrid model. *Neurocomputing* 203 (2016), 22–33.
[18] Matthias Fey and Jan E. Lenssen. 2019. Fast graph representation learning with PyTorch geometric. In *Proceedings of the ICLR Workshop on Representation Learning on Graphs and Manifolds*.
[19] Maxi Fischer, Willi Gierke, Thomas Kellermeier, Ajay Kesar, Axel Stebner, and Daniel Thevessen. 2019. Anomaly Detection on Time Series: An Evaluation of Deep Learning Methods. Retrieved from https://github.com/KDD-OpenSource/DeepADoTS.
[20] Donald Goldfarb and Zhiwei Qin. 2014. Robust low-rank tensor recovery: Models and algorithms. *SIAM J. Matrix Anal. Appl.* 35, 1 (2014), 225–253.
[21] Palash Goyal, Nitin Kamra, Xinran He, and Yan Liu. 2018. Dyngem: Deep embedding method for dynamic graphs. Retrieved from https://arXiv:1805.11273.
[22] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*. MIT Press, 1024–1034.
[23] Simon Hawkins, Hongxing He, Graham Williams, and Rohan Baxter. 2002. Outlier detection using replicator neural networks. In *Proceedings of the International Conference on Data Warehousing and Knowledge Discovery*. Springer, 170–180.
[24] Yue Hu and Daniel B. Work. 2020. Robust tensor recovery with fiber outliers for traffic events. *ACM Trans. Knowl. Discov. Data* 15, 1 (2020), 1–27.
[25] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR'15)*.
[26] Thomas N. Kipf and Max Welling. 2016. Variational graph auto-encoders. Retrieved from https://arXiv:1611.07308.
[27] Thomas N. Kipf and Max Welling. 2017. Semi-supervised classification with graph convolutional networks. In *Proceedings of the International Conference on Learning Representations (ICLR'17)*.
[28] Prakhyat G. Kulkarni, S. Y. Praneet, R. B. Raghav, and Bhaskarjyoti Das. 2020. Deep detection of anomalies in static attributed graph. In *Proceedings of the International Conference on Machine Learning, Image Processing, Network Security and Data Sciences*. Springer, 627–640.
[29] Atsutoshi Kumagai, Tomoharu Iwata, and Yasuhiro Fujiwara. 2020. Semi-supervised anomaly detection on attributed graphs. Retrieved from https://arXiv:2002.12011.
[30] Duo Li and Joan Lasenby. 2021. Spatiotemporal attention-based graph convolution network for segment-level traffic prediction. *IEEE Trans. Intell. Transport. Syst.* (2021), 1–9. https://doi.org/10.1109/TITS.2021.3078187
[31] Yuening Li, Xiao Huang, Jundong Li, Mengnan Du, and Na Zou. 2019. SpecAE: Spectral AutoEncoder for anomaly detection in attributed networks. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*. 2233–2236.
[32] Zhishuai Li, Gang Xiong, Yuanyuan Chen, Yisheng Lv, Bin Hu, Fenghua Zhu, and Fei-Yue Wang. 2019. A hybrid deep learning approach with GCN and LSTM for traffic flow prediction. In *Proceedings of the IEEE Intelligent Transportation Systems Conference (ITSC'19)*. 1929–1933. https://doi.org/10.1109/ITSC.2019.8916778

[33] Pankaj Malhotra, Anusha Ramakrishnan, Gaurangi Anand, Lovekesh Vig, Puneet Agarwal, and Gautam Shroff. 2016. LSTM-based encoder-decoder for multi-sensor anomaly detection. Retrieved from https://arXiv:1607.00148.

[34] Deborah Matherly, Neeli Langdon, Brian Wolshon, Pamela Murray-Tuite, John Renne, Roberta Thomas, Jane Mobley, and Kelly Reinhardt. 2014. *A Guide to Regional Transportation Planning for Disasters, Emergencies, and Significant Events*. National Cooperative Highway Research Program (NCHRP) Report 777, Project No. NCHRP 20-59(42). TRB Publications.

[35] Abduallah Mohamed, Kun Qian, Mohamed Elhoseiny, and Christian Claudel. 2020. Social-STGCNN: A social spatio-temporal graph convolutional neural network for human trajectory prediction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 14424–14432.

[36] Nashville. 2019. 2019 NFL Draft Event Schedule. Retrieved from https://www.visitmusiccity.com/nfldraft/nfl-draft-event-schedule.

[37] Hao Peng, Bowen Du, Mingsheng Liu, Mingzhe Liu, Shumei Ji, Senzhang Wang, Xu Zhang, and Lifang He. 2021. Dynamic graph convolutional network for long-term traffic flow prediction with reinforcement learning. *Info. Sci.* 578 (2021), 401–416. https://doi.org/10.1016/j.ins.2021.07.007

[38] Mayu Sakurada and Takehisa Yairi. 2014. Anomaly detection using autoencoders with nonlinear dimensionality reduction. In *Proceedings of the MLSDA 2nd Workshop on Machine Learning for Sensory Data Analysis*. 4–11.

[39] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne Van Den Berg, Ivan Titov, and Max Welling. 2018. Modeling relational data with graph convolutional networks. In *Proceedings of the European Semantic Web Conference*. Springer, 593–607.

[40] Yunsheng Shi, Zhengjie Huang, Shikun Feng, Hui Zhong, Wenjin Wang, and Yu Sun. 2020. Masked label prediction: Unified message passing model for semi-supervised classification. Retrieved from https://arXiv:2009.03509.

[41] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. Dropout: A simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.* 15, 1 (2014), 1929–1958.

[42] Ya Su, Youjian Zhao, Chenhao Niu, Rong Liu, Wei Sun, and Dan Pei. 2019. Robust anomaly detection for multivariate time series through stochastic recurrent neural network. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2828–2837.

[43] Huachun Tan, Jianshuai Feng, Guangdong Feng, Wuhong Wang, and Yu-Jin Zhang. 2013. Traffic volume data outlier recovery via tensor model. *Math. Problems Eng.* 2013 (2013).

[44] TDOT. 2019. TDOT Weekly Construction Report for Middle Tennessee. Retrieved from https://preprod.tn.gov/tdot/news/2019/4/17/tdot-weekly-construction-report-for-middle-tennessee--april-18-24--2019.html.

[45] Fei Tian, Bin Gao, Qing Cui, Enhong Chen, and Tie-Yan Liu. 2014. Learning deep representations for graph clustering. In *Proceedings of the Association for the Advancement of Artificial Intelligence (AAAI'14)*, Vol. 14. 1293–1299.

[46] NYC TLC. 2020. TLC Trip Record Data. Retrieved from https://www1.nyc.gov/site/tlc/index.page.

[47] Uber. 2020. Uber Movement. Retrieved from https://movement.uber.com/.

[48] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. Retrieved from https://arXiv:1710.10903.

[49] Chun Wang, Shirui Pan, Guodong Long, Xingquan Zhu, and Jing Jiang. 2017. Mgae: Marginalized graph autoencoder for graph clustering. In *Proceedings of the ACM on Conference on Information and Knowledge Management*. 889–898.

[50] Meng-Xiang Wang, Wang-Chien Lee, Tao-Yang Fu, and Ge Yu. 2020. On representation learning for road networks. *ACM Trans. Intell. Syst. Technol.* 12, 1 (2020), 1–27.

[51] Yuandong Wang, Hongzhi Yin, Hongxu Chen, Tianyu Wo, Jie Xu, and Kai Zheng. 2019. Origin-destination matrix prediction via graph convolution: A new perspective of passenger demand modeling. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 1227–1235.

[52] Yuandong Wang, Hongzhi Yin, Tong Chen, Chunyang Liu, Ben Wang, Tianyu Wo, and Jie Xu. 2021. Gallat: A spatiotemporal graph attention network for passenger demand prediction. In *Proceedings of the IEEE 37th International Conference on Data Engineering (ICDE'21)*. 2129–2134. https://doi.org/10.1109/ICDE51399.2021.00212

[53] Zihan Wang, Zhaochun Ren, Chunyu He, Peng Zhang, and Yue Hu. 2019. Robust embedding with multi-level structures for link prediction. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI'19)*. 5240–5246.

[54] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. 2019. Simplifying graph convolutional networks. In *Proceedings of the International Conference on Machine Learning*. PMLR, 6861–6871.

[55] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and S. Yu Philip. 2020. A comprehensive survey on graph neural networks. *IEEE Trans. Neural Netw. Learn. Syst.* 32, 1 (2020), 4–24.

[56] Haowen Xu, Wenxiao Chen, Nengwen Zhao, Zeyan Li, Jiahao Bu, Zhihan Li, Ying Liu, Youjian Zhao, Dan Pei, Yang Feng et al. 2018. Unsupervised anomaly detection via variational auto-encoder for seasonal KPIs in web applications. In *Proceedings of the World Wide Web Conference*. 187–196.

[57] Bishan Yang, Wen-tau Yih, Xiaodong He, Jianfeng Gao, and Li Deng. 2014. Embedding entities and relations for learning and inference in knowledge bases. Retrieved from https://arXiv:1412.6575.

[58] Shiming Yang, Konstantinos Kalpakis, and Alain Biem. 2014. Detecting road traffic events by coupling multiple time-series with a nonparametric bayesian method. *IEEE Trans. Intell. Transport. Syst.* 15, 5 (2014), 1936–1946.

[59] Bing Yu, Haoteng Yin, and Zhanxing Zhu. 2017. Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting. *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI'17)*.

[60] Bing Yu, Haoteng Yin, and Zhanxing Zhu. 2018. Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI'18)*. International Joint Conferences on Artificial Intelligence Organization, 3634–3640. https://doi.org/10.24963/ijcai.2018/505

[61] Wenchao Yu, Wei Cheng, Charu C. Aggarwal, Kai Zhang, Haifeng Chen, and Wei Wang. 2018. Netwalk: A flexible deep embedding approach for anomaly detection in dynamic networks. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2672–2681.

[62] Shuangfei Zhai, Yu Cheng, Weining Lu, and Zhongfei Zhang. 2016. Deep structured energy based models for anomaly detection. In *Proceedings of the 33rd International Conference on International Conference on Machine Learning-Volume 48*. 1100–1109.

[63] Xiyue Zhang, Chao Huang, Yong Xu, and Lianghao Xia. 2020. Spatial-temporal convolutional graph attention networks for citywide traffic flow forecasting. In *Proceedings of the 29th ACM International Conference on Information and; Knowledge Management (CIKM'20)*. ACM, New York, NY, 1853–1862. https://doi.org/10.1145/3340531.3411941

[64] Zikai Zhang, Yidong Li, Haifeng Song, and Hairong Dong. 2021. Multiple dynamic graph based traffic speed prediction method. *Neurocomputing* 461 (2021), 109–117. https://doi.org/10.1016/j.neucom.2021.07.052

[65] Li Zheng, Zhenpeng Li, Jian Li, Zhao Li, and Jun Gao. 2019. AddGraph: Anomaly detection in dynamic graph using attention-based temporal GCN. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI'19)*. 4419–4425.

[66] Chong Zhou and Randy C. Paffenroth. 2017. Anomaly detection with robust deep autoencoders. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 665–674.

[67] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. 2020. Graph neural networks: A review of methods and applications. *AI Open* 1 (2020), 57–81.

[68] Le-kui Zhou, Yang Yang, Xiang Ren, Fei Wu, and Yueting Zhuang. 2018. Dynamic network embedding by modeling triadic closure process. In *Proceedings of the Association for the Advancement of Artificial Intelligence (AAAI'18)*. 571–578.

[69] Dali Zhu, Yuchen Ma, and Yinlong Liu. 2020. DeepAD: A joint embedding approach for anomaly detection on attributed networks. In *Proceedings of the International Conference on Computational Science*. Springer, 294–307.

[70] Bo Zong, Qi Song, Martin Renqiang Min, Wei Cheng, Cristian Lumezanu, Daeki Cho, and Haifeng Chen. 2018. Deep autoencoding gaussian mixture model for unsupervised anomaly detection. In *Proceedings of the International Conference on Learning Representations*.

Received July 2021; revised March 2022; accepted May 2022